

Digital Control III: Deadbeat Control

A discrete-only trick: reach the target in a finite number of steps.

In continuous time you cannot place a pole "at $-\infty$ ", so a response only ever decays asymptotically. In discrete time you can place **every** closed-loop pole at $z = 0$: the closed-loop state-transition matrix becomes nilpotent, and the system reaches its target in at most n samples. This is **deadbeat** control. You will:

- discretize a plant and place its discrete poles at the origin with **acker**,
- verify the closed-loop $A - BK$ is nilpotent, and
- watch the state hit zero in exactly n steps.

Run with `publish('DeadbeatControl.m')`, or step through with **Ctrl+Enter**.

Contents

- Discretize the plant
- Place all discrete poles at $z = 0$
- Nilpotency: why it settles in n steps
- Before vs. after: open loop vs. deadbeat
- The price of deadbeat: control effort
- Summary
- Try it yourself

Discretize the plant

Continuous plant $G(s) = \frac{1}{s^2 + s}$ (an integrator plus a lag), sampled at $T = 0.2$ s.

```
G = tf(1,[1 1 0]);
T = 0.2;
sysd = c2d(ss(G), T);
A = sysd.A; B = sysd.B;
n = size(A,1);
```

Place all discrete poles at $z = 0$

Deadbeat design puts every eigenvalue of $A - BK$ at the origin.

```
K = acker(A, B, zeros(1,n));
fprintf('Deadbeat gain K = %s\n', mat2str(K,4))
fprintf('Closed-loop poles: %s (all at 0)\n', mat2str(eig(A-B*K).',3))
```

```
Deadbeat gain K = [7.183 27.58]
Closed-loop poles: [-1.71e-08 1.71e-08] (all at 0)
```

Nilpotency: why it settles in n steps

Because the closed-loop matrix has all eigenvalues at 0, it is **nilpotent**: $(A - BK)^n = 0$. From any initial state the free response is therefore identically zero after n samples.

```
Acl = A - B*K;
fprintf('||(A-BK)^%d|| = %.2e (should be ~0)\n', n, norm(Acl^n))
```

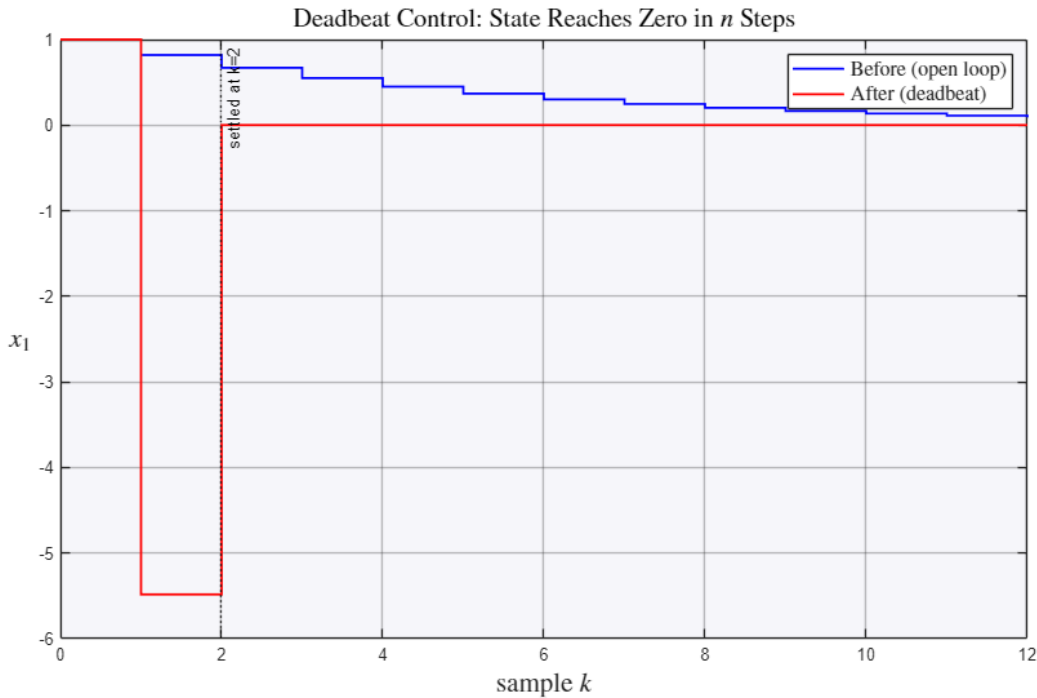
```
||(A-BK)^2|| = 1.84e-15 (should be ~0)
```

Before vs. after: open loop vs. deadbeat

Simulate both from the same initial state. The open-loop plant (a pole at $z = 1$) never returns to zero; the deadbeat loop snaps to zero at $k = n$.

```
N = 12;
x_ol = zeros(n,N+1); x_ol(:,1) = [1;1];
x_db = zeros(n,N+1); x_db(:,1) = [1;1];
for k = 1:N
    x_ol(:,k+1) = A * x_ol(:,k);
    x_db(:,k+1) = Acl * x_db(:,k);
end

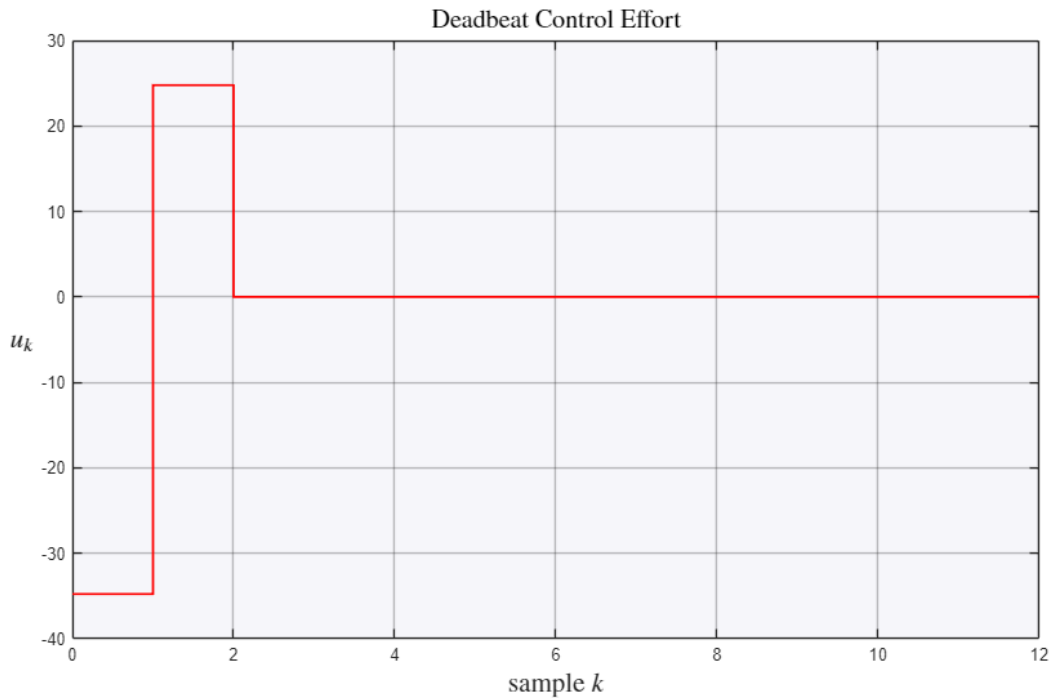
figure
stairs(0:N, x_ol(1,:), 'b', 'LineWidth', 1.3)
hold on
stairs(0:N, x_db(1,:), 'r', 'LineWidth', 1.3)
xline(n, 'k:', sprintf('settled at k=%d', n))
hold off
grid on
legend('Before (open loop)', 'After (deadbeat)', 'Interpreter', 'latex', 'FontSize', 12)
title('Deadbeat Control: State Reaches Zero in $n$ Steps', 'Interpreter', 'latex', 'FontSize', 15)
ylabel('$x_1$', 'Interpreter', 'latex', 'FontSize', 16); set(get(gca, 'YLabel'), 'Rotation', 0)
xlabel('sample $k$', 'Interpreter', 'latex', 'FontSize', 16)
```



The price of deadbeat: control effort

Settling instantly is not free – the control signal $u_k = -Kx_k$ is large at the first step, and the design is very sensitive to the model and the chosen sample time.

```
u_db = -K * x_db;
figure
stairs(0:N, u_db, 'r', 'LineWidth', 1.3)
grid on
title('Deadbeat Control Effort', 'Interpreter', 'latex', 'FontSize', 15)
ylabel('$u_k$', 'Interpreter', 'latex', 'FontSize', 16); set(get(gca, 'YLabel'), 'Rotation', 0)
xlabel('sample $k$', 'Interpreter', 'latex', 'FontSize', 16)
```



Summary

- Discrete pole placement at $z = 0$ makes $A - BK$ nilpotent.
- The closed loop settles **exactly**, in at most n samples – a feat impossible in continuous time.
- The cost is large control effort and high sensitivity, so deadbeat is used sparingly and only when the model is trustworthy.

Try it yourself

- Lengthen the sample time T and notice the deadbeat gain K shrink, while the fixed n -step settling takes longer in real time.
- Add a third state and confirm the loop still settles in exactly n steps with $(A - BK)^n = 0$.